

LV09 포인터 복습, 구조체 포인터 변수

■ 소유자	 암암코딩
■ 태그	
■ 생성 일시	@2024년 2월 3일 오후 7:26

<https://youtu.be/wVIGmqSroJ0?feature=shared>

<https://youtu.be/CZRLEPhYPRM?feature=shared>

<https://youtu.be/9XM5HqiXKTA?feature=shared>

포인터 타입 변수와 동적 할당

포인터란?

🎯 주소 값을 저장할 수 있는 특별한 타입의 변수

- 숫자를 저장하고 싶다 → `int` 타입 변수를 사용한다
- 글자를 저장하고 싶다 → `char` 타입 변수를 사용한다
- 주소 값을 저장할 수 있는 타입을 포인터 타입 변수라고 한다

```
int main()
{
    // 📍 포인터 타입 목적

    unsigned int addressNumber = 0;
```

// 주소는 숫자니까 사실 안 됨에 저장해도 상관이 없다.

```
int a = 0;
```

```
addressNumber = (unsigned int)&a; // 강제 형변환으로 주소를 숫자로
```

```
// 💡 포인터 변수에 * 을 정에서
```

```
// 해당 주소에 위치(변수)를 이동할 수 있다.
```

```
// 또는 가르킨다 라고 한다.
```

```
int* p = &a; // p는 a의 주소를 저장
```

```
*p = 100; // p가 가리키는 곳(a)에 100 저장
```

```
return 0;
```

```
}
```

☁ 포인터 동작 원리:

메모리 상황:

└─ 변수 a: 주소 0x1000, 값 0

└─ 포인터 p: 주소 0x2000, 값 0x1000 (a의 주소)

└─ *p = 100 실행 → 0x1000 주소(a)에 100 저장

포인터 배열 vs 배열 포인터

포인터 배열 (Pointer Array)

📦 포인터들의 배열 - 여러 개의 포인터를 담는 배열

```
// 포인터 배열
```

```
int a, b, c;
```

```
int* pArr[10] = { &a, &b, &c }; // 포인터를 담는 배열
```

배열 포인터 (Array Pointer)

🎯 배열을 가리키는 포인터 - 배열 전체를 가리키는 포인터

```
// 배열 포인터
```

```
int arr[10] = { 1,2,3,4,5,6,7,8,9,10 };
```

```
int (*pArr2)[10] = &arr; // 배열을 가리키는 포인터
```

🔍 구분 방법:

- `int* pArr[10]` : `[]` 가 우선순위 높음 → 포인터 배열
- `int (*pArr2)[10]` : `()` 로 묶어서 포인터 먼저 → 배열 포인터

구조체 포인터와 -> 연산자

구조체, 클래스 타입의 변수도 결국 자료형이기 때문에 포인터 타입으로 선언이 가능하다.

```
#include <iostream>
```

```
struct Vector2
{
    int x;
    int y;
};
```

```
int main()
{
```

```
    // 구조체 포인터 선언 목적
```

```
    Vector2 p1;
```

```
    Vector2* vp = &p1;
```

```
    // 🚫 방법 1: (*포인터).멤버 - 번거로움
```

```
    (*vp).x = 100;
```

```
    (*vp).y = 100;
```

```
    //*(vp.x) = 200; // ❌ 오류!
```

```
    // 연산자가 역산자보다 우선순위가 높기 때문에
```

```
    // 제대로 구조체 변수에 접근할 수 없다.
```

```
    // ✅ 방법 2: -> 포인터 연산자 (깔끔!)
```

```
    vp->x = 100;
```

```
    vp->y = 100;
```

```
return 0;
}
```

💡 -> 연산자의 편리함:

- `(*vp).x` = `vp->x` (같은 의미, 더 간단한 표현)

연결리스트 구현 예시

위에서 배운 내용을 활용하기 위해서 다음과 같이

구조체 포인터 변수와 같은 구조체의 주소를 저장할 수 있는 포인터타입을 선언하여 각 객체를 연결시켜 보자.

```
#include <iostream>

struct Node
{
    int data;    // 데이터 저장
    Node* next; // 다음 노드를 가리키는 포인터
};

int main()
{
    // 📝 종이 할 것이 대우 떡리해진다.
    Node* head; // 첫 번째 노드를 가리키는 포인터
    Node n1;    // 첫 번째 노드
    Node n2;    // 두 번째 노드
    Node n3;    // 세 번째 노드

    // 🔗 노드들을 연결
    head = &n1;    // head가 첫 번째 노드를 가리킴
    n1.next = &n2; // 첫 번째 → 두 번째
    n2.next = &n3; // 두 번째 → 세 번째

    // 📊 데이터 저장
    n1.data = 1;
    n2.data = 2;
    n3.data = 3;
```

```
// 🖨️ 연결된 데이터 출력
std::cout << head->data;           // 1 출력
std::cout << head->next->data;      // 2 출력
std::cout << head->next->next->data; // 3 출력

return 0;
}
```

🔗 연결리스트 동작 과정:

```
head → [1|next] → [2|next] → [3|null]
      ↑ n1   ↑ n2   ↑ n3
```

```
head->data: 1
head->next->data: 2
head->next->next->data: 3
```

🎯 핵심 포인트:

- **포인터**: 주소를 저장하는 변수 (`int* p`)
- **역참조**: `p` 로 포인터가 가리키는 값에 접근
- **구조체 포인터**: `>` 연산자로 멤버에 간편하게 접근
- **연결리스트**: 포인터로 노드들을 체인처럼 연결하는 자료구조

💡 동적할당

🔄 **동적 메모리 할당**이란 컴퓨터 프로그래밍에서 실행 시간 동안 사용할 메모리 공간을 할당하는 것을 말한다. 사용이 끝나면 해제하여야 한다.

📋 특징:

- **정적 할당**: 컴파일 시간에 메모리 크기 결정
- **동적 할당**: 실행 시간에 메모리 크기 결정
- **메모리 해제**: 프로그래머가 명시적으로 해제하거나 가비지 컬렉션 대상

🎯 **장점**: 필요한 만큼만 메모리 사용 가능

⚠️ **주의**: 사용 후 반드시 메모리 해제 필요

"강의는 많은데, 왜 나는 아직도 코드를 못 짤까?"

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 **손이 안 움직이고**,
- 복습을 하려 해도 **무엇을 다시 봐야 할지 모르겠고**,
- 질문할 곳도 없고,
- 유튜브는 결국 **정답을 따라 치는 것밖에 안 되는 것 같고**.

문제는 '연습'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 얀얀코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얀얀코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 연습문제를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 **스스로 쪼개고 설계하는 힘**
- 다양한 조건을 만족시키는 **실제 구현 능력**
- 기능 단위가 아닌, **프로그램 단위로 사고하는 습관**
- 마침내 **자신의 힘으로 코드를 끝까지 작성하는 경험**

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얀얀코딩 상담방](#)

▼ oopy:hide

연습문제

이제부터는 많은 자료구조를 배우기 시작합니다.

대표적인 자료구조인 링크드리스트로 시작을 합니다.

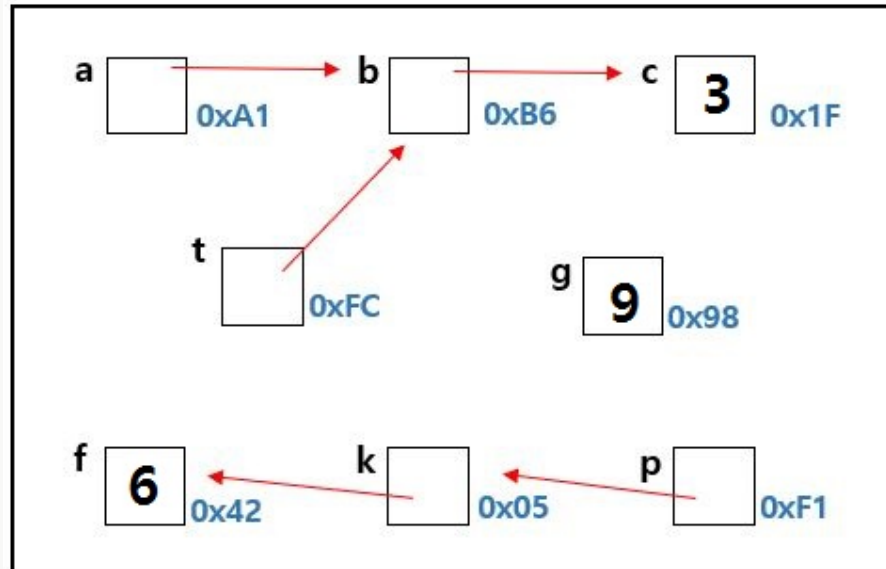
이제, 구조체와 포인터를 활용하여 코딩 할 때가 되었습니다.



널가리키며

문제 1번

아래그림과 같이 구성되어 있는 변수들이 있습니다.



다음 소스코드를 copy해서 **네모칸**에 알맞은 답을 적어 제출하세요.
(16진수는 대문자로 입력해주세요)

```
#include <iostream>

using namespace std;

int main()
{

    cout << "0xB6" << endl; // a의 값 <--- 예시정답

    cout << "0x□" << endl; // k의 값

    cout << "0x□" << endl; // t의 값

    cout << "0x□" << endl; // c의 주소
```

```
cout << "0x□" << endl; // *a

cout << "□" << endl; // **p

cout << "0x□" << endl; // &k

cout << "0x□" << endl; // *p

return 0;

}
```



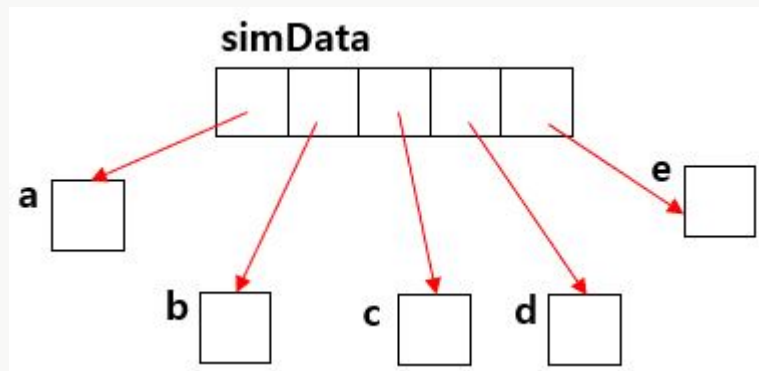
아침드라마 심슨

문제 2번



인기쟁이 심슨은 다섯명과 연애했습니다.

a ~ e 까지의 변수에 그녀들의 나이를 입력 받아주세요.



그리고 for문을 돌려

가장 '나이가 많은 사람'과 '어린 사람의 나이'를 찾아 출력해주세요.

simData는 포인터 배열이고, 각각 **a, b, c, d, e** 를 가리켜야 합니다.

입력 예제

35

72

40

25

120

출력 결과

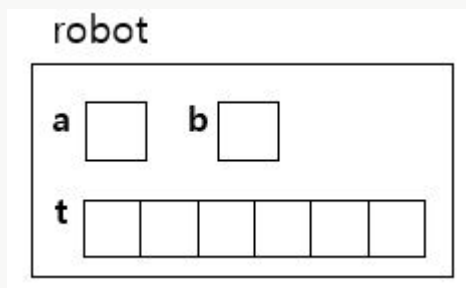
MAX:120

MIN:25



내이름은 로버트

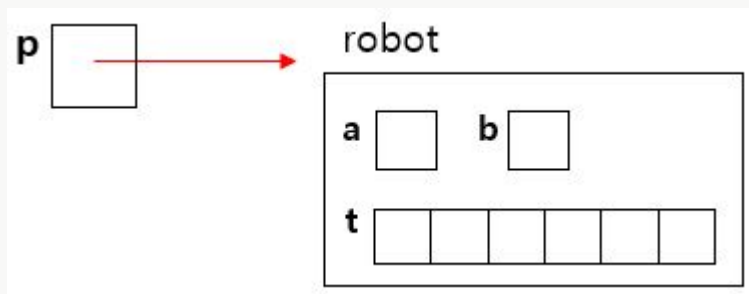
문제 3번



robot 구조체를 정의하고, 구체체 변수 **robert**를 만들어주세요.

이곳에 들어갈 숫자 2개와 문장 1개를 입력 받아주세요.

그리고 **robert**를 가르키는 포인터 1개를 만들어서 가리키도록 해주세요.



이제 포인터 **p**를 이용해서 **a+b**값과 문장**t**를 출력하세요.

입력 예제

3 5 super

출력 결과

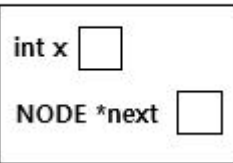
8 super



간단한 링크드리스트 만들기

문제 4번

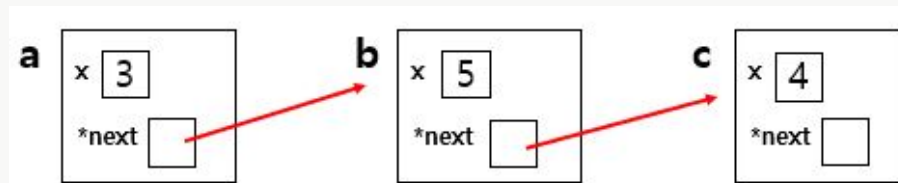
NODE



위와 같은 노드를 만들어주세요.

그리고 노드의 변수 3개 a, b, c 를 만들어 주세요.

이제 **기호 ->** 를 이용해서 다음과 같이 구성하고 값을 채워주세요



(입력과 출력은 없습니다)



심슨의 관계도

문제 5번

NODE

age

next1

next2

를 만들어 주세요.

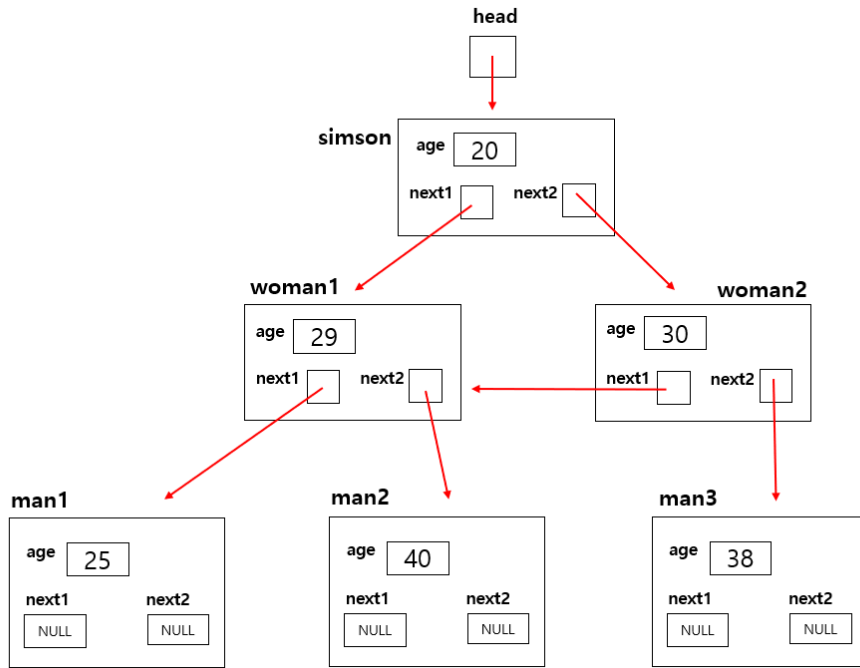


심슨은 알고보니 피해자였습니다.

그녀들은 심슨을 이용하고 있었습니다.

다음과 같은 관계 구조를 만들어주세요.

(입력과 출력은 없습니다)



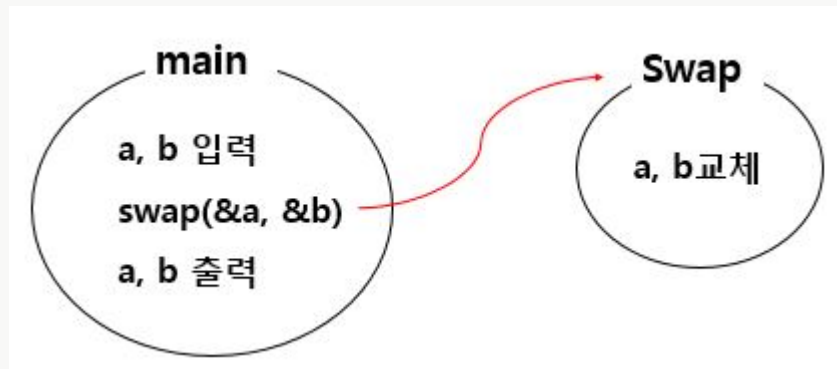


swap 함수로 포인터 떠올리기

문제 6번

변수 **a, b**에다가 숫자를 입력 받으세요.

그리고 변수 a와 b의 주소를 보내 swap을 해주는 함수를 만들어서 swap 후 출력 해주세요.



ex)

입력: 3 5

출력: 5 3

입력 예제

3 5

출력 결과

5 3



중복을 찾아라

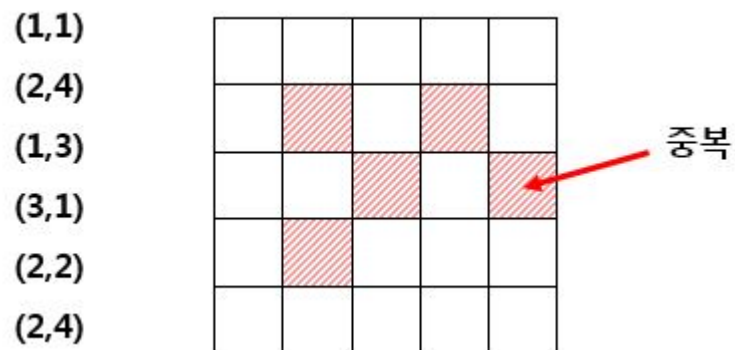
문제 7번

좌표 6개를 입력 받으세요.

이 좌표중 중복된 좌표가 있는지 없는지 존재여부를 출력 해주세요.

중복된 좌표가 있으면 "중복된좌표발견", 없으면 "중복없음"으로 출력

ex)



결과 : 중복된좌표발견

입력 예제

1 1

2 4

1 3

3 1

2 2

2 4

출력 결과

중복된좌표발견



원하는 범위의 값 반대로 출력하기

문제 8번

0	1	2	3	4	5	6
3	5	5	6	9	1	2

위 배열을 하드코딩해주세요.

숫자의 범위를 입력 받으세요(숫자 2개 입력)

그리고 이 범위의 값들을 반대로 돌려 출력해주는 프로그램을 만들어 주세요.

만약 3 5를 입력 받았다면

3번 index ~ 5번 index 사의 값만 반대로 돌려주세요.

아래 그림을 참고하여 프로그램을 만들어 주세요.

ex)

입력 : 3 5



출력: 3 5 5 1 9 6 2

입력 예제

3 5

출력 결과

3 5 5 1 9 6 2



철수와 영철이의 승부수

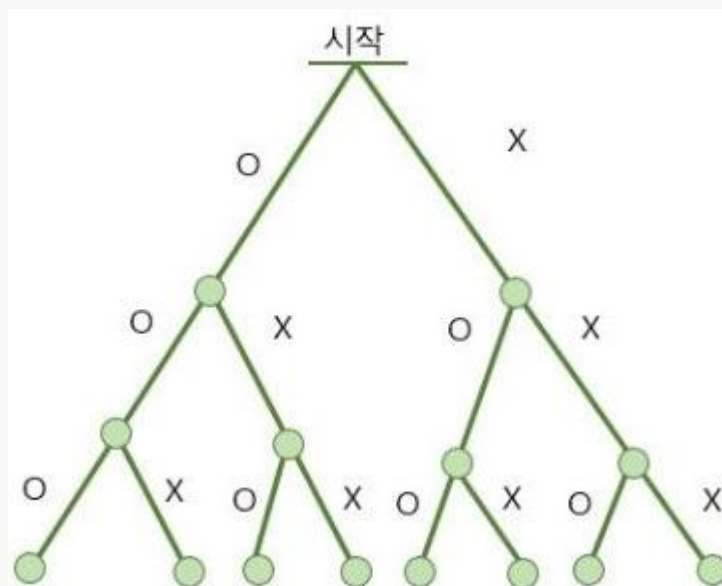
문제 9번

철수와 영철이는 3번의 게임을 하려고 합니다.

아래와 같이 승패의 모든 경우를 트리로 그려 보았습니다.

이 트리를 구현하여 철수가 영철이에게 3번의 게임에서 승리하고 패하는 모든 경우를 출력 해주세요.

HINT: OX로 구성된 path 전역배열을 이용해서 승패를 출력할 수 있습니다.



출력 결과

3승(승승승)

2승1패(승승패)

2승1패(승패승)

1승2패(승패패)

2승1패(패승승)

1승2패(패승패)

1승2패(패패승)

3패(패패패)



난 너를 택했다

문제 10번

한 문장을 입력받아 str 배열에 입력 받으세요.

index 숫자 4개를 입력 받아 data 배열에 저장하세요.

네 칸짜리 포인터 배열 ptrs를 만들고

str 배열 안에서 입력받은 index를 각각 가리켜주세요.

그리고 ptrs에서 가리키는 값들은 for문을 돌려 출력하세요.

ex)

BKABCDE 을 입력했다면
0 1 5 6

ptrs



strs



이렇게 연결시키면 됩니다.
그리고 출력결과는 **BKDE**

ex)

ABCDEFGFG 를 입력했다면 출력결과는 **CDEE**
2 3 4 4

입력 예제

BKABCDE

0 1 5 6

출력 결과

BKDE



단단한 성

문제 11번

아래 그림을 보고 규칙을 찾아내어 숫자성이

"안전한성"인지 "안전하지않은성" 인지 출력 해주는 프로그램을 만들어주세요.

ex)

아래는 안전하게 성이 쌓여져 있는 모습입니다.

각 숫자는 돌의 단단함을 나타내는 수치입니다.

그러나 만약, 숫자2 위에 숫자3이 올려져 있다면 이 성은 무너질 것 입니다.

	1	1	
	2	2	1
2	3	3	2
3	3	3	2

아래의 배열을 하드코딩하여 안전한성인지 아닌지 판별하는 프로그램을 작성해주세요.

결과 : 안전하지않은성

	1	1	
2	2	3	
1	3	3	1

출력 결과

안전하지않은성

